STACKS LIBRARY MANAGEMENT SYSTEM

Robotic Process Automation (RPA) Project Documentation & Workflow Manual

---

A complete technical walkthrough of desk circulation, member management, and autonomous background robotic workflows.

System Type: Web-Based Desk Application with Integrated RPA Subsystem

Core Frameworks: Python Flask, SQLite, Robot Framework

Audience: Project Evaluation Committee / Viva Examination

Format: High-Contrast Monochrome Technical Specification

1. Project Scope & Overview

The **Stacks Library Management System** is an integrated college library desk application designed to eliminate manual, error-prone administrative tasks through **Robotic Process Automation (RPA)**.

1.1 Problem Statement

Traditional library management involves highly repetitive manual interventions: tracking loan return deadlines, calculating overdue penalty fines day-by-day, sending individual warning emails to students, and auditing shelf inventory to identify depleted book copies. Manual handling leads to missed notices, inaccurate fine collections, and delayed stock replenishments.

1.2 Project Scope & Objectives

- **Core Desk Operations:** Maintain full catalogue control (ISBN, author, title, category, copies), student membership records, and a formal lending desk workflow for book checkout and returns.
- **Role-Based Security:** Strict multi-role separation among `Admin`, `Librarian`, and `Student` users.
- **Autonomous RPA Integration:** Scheduled, headless software bots that autonomously monitor loan deadlines, calculate monetary fines, send email reminders, and audit inventory health without requiring continuous manual staff effort.
- **Dual-Log Audit Architecture:** Total separation between bot execution telemetry (`automation_logs`) and human staff audit tracking (`user_activity_log`).

1.3 Technical Architecture

Component	Technology	Description & Purpose
Backend API	Python 3.12 / Flask 3.1.3	RESTful endpoints, session security, business logic, fine calculations.
Database	SQLite 3 (`library.db`)	Relational storage: books, members, loans, users, automation_logs, activity logs.
RPA Engine	Robot Framework 7.4.2	Autonomous bot test suites driving authenticated API workflows.
Notification Layer	Python `email_service`	MIME messaging with pluggable SMTP, Console, and Mock transports.
User Interface	Vanilla HTML5, CSS3, JS	Fast, responsive single-page desk interface with modal dialogs and live badges.

2. Written Workflow in Simple Language

The entire system operates through a sequential, logical lifecycle from initial book procurement to automated loan recovery:

Step 1: Adding Books to the Catalogue
A librarian or administrator enters book details (Title, Author, ISBN, Category, Total Copies). The system automatically validates the ISBN for uniqueness and initializes available copies equal to total copies.

Step 2: Member Registration & Activation
Students can self-register via the online registration form (`/register`). Alternatively, desk staff can register members in person. Staff-created members subsequently use the activation screen (`/activate`) to set their password and gain student portal access.

Step 3: Book Checkout (Lending Desk)
Staff select an available book and a registered member. The system checks copy availability, decrements available copies by 1, creates an active loan record, and automatically assigns a due date.

Step 4: Proactive Due-Date Monitoring (Courtesy RPA Bot)
Before any book becomes overdue, the automated **Courtesy Due-Date Reminder Bot** scans all active loans due within the next 24 to 48 hours. It dispatches a polite reminder email to the borrower, allowing them to return or renew the book before incurring fines.

Step 5: Overdue Detection & Fine Calculation (Overdue RPA Bot)
When a loan crosses its due date without being returned, the automated **Overdue Reminder Bot** triggers. It calculates elapsed overdue days, assesses an automatic penalty fine of ₹2.00 per day, records the result in the automation log, and sends an overdue warning notice to the borrower.

Step 6: Book Return & Fine Settlement
When the student returns the book to the desk, staff click "Return". The system increments available copies back by 1 and finalizes the fine amount. When the fine is settled, staff click "Mark Paid", clearing the student's dues.

Step 7: Automated Stock & Circulation Auditing (Inventory RPA Bot)
Staff run the **Inventory Health Audit Bot** to inspect catalogue circulation. The bot identifies completely depleted titles (0 copies available), flags low stock, computes utilization rates, and generates an automated Restock Advisory CSV file.

3. Page-by-Page Walkthrough & Screenshots

The following section illustrates every primary interface in the system with a concise summary of its functional purpose.

3.1 Login Screen (`/login`)

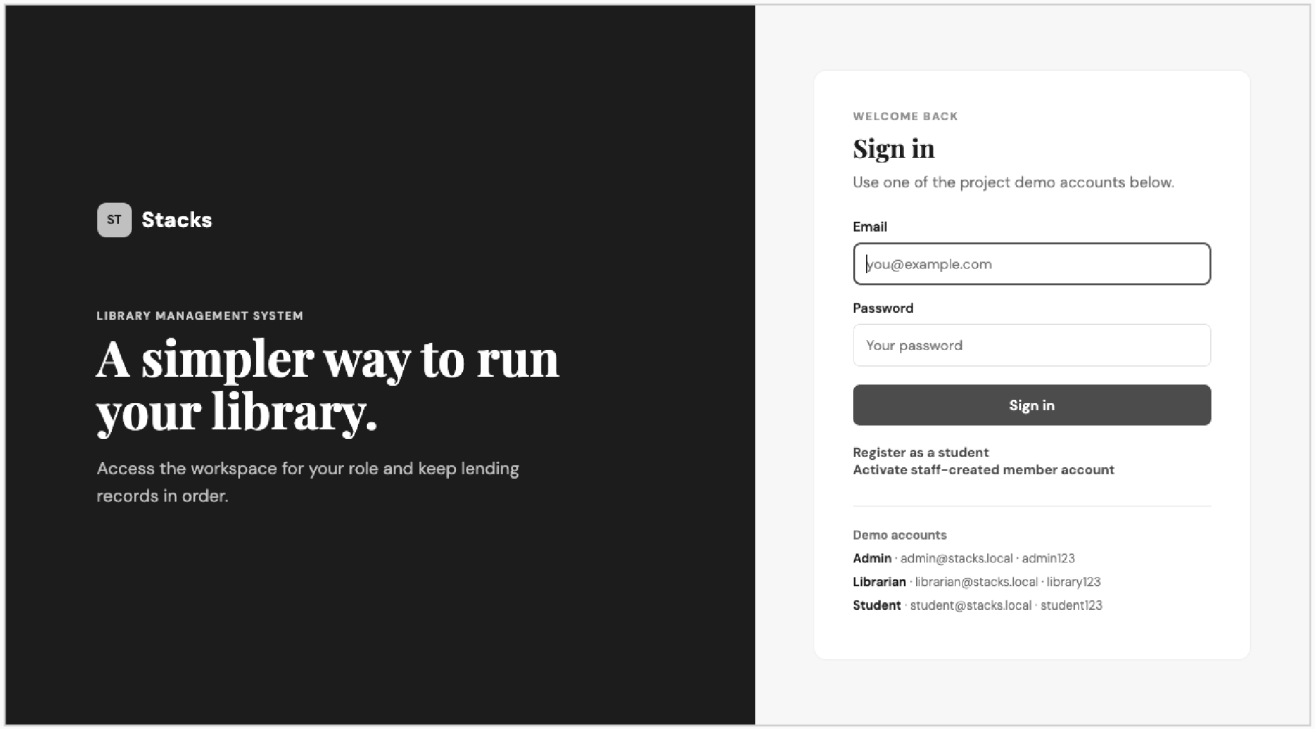


Figure 3.1: Centralized Secure Authentication Portal

Allows Admin, Librarian, and Student users to authenticate using hashed credentials, routing them to role-specific capabilities.

3.2 Student Self-Registration (`/register`)

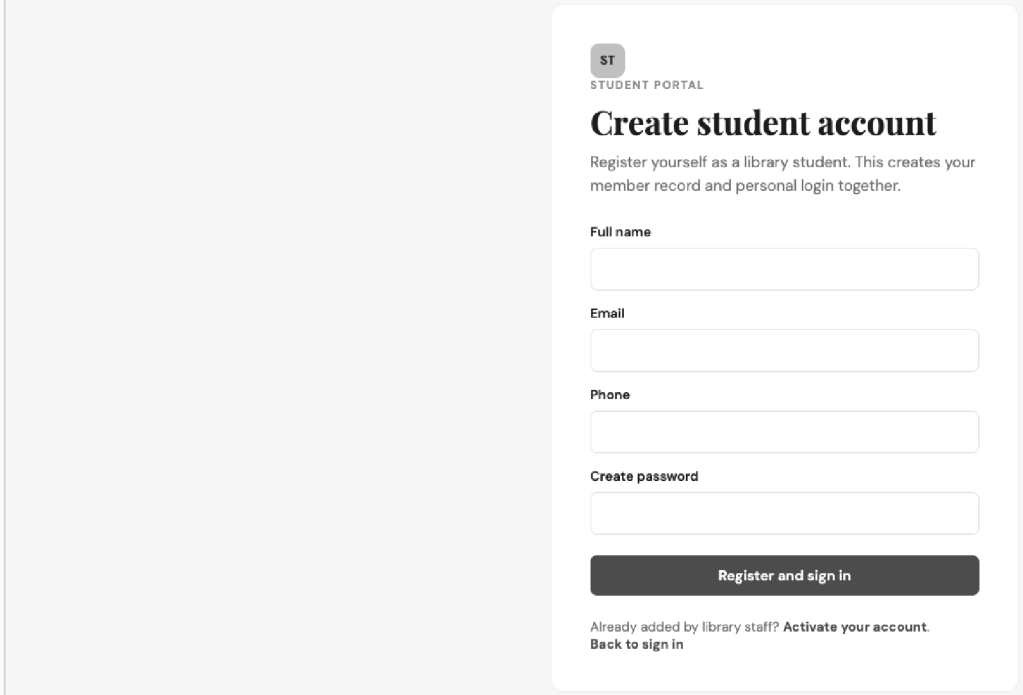


Figure 3.2: Student Self-Service Registration Page

Enables new students to register their membership and online login simultaneously with unique email verification.

3.3 Staff-Created Member Activation (`/activate`)

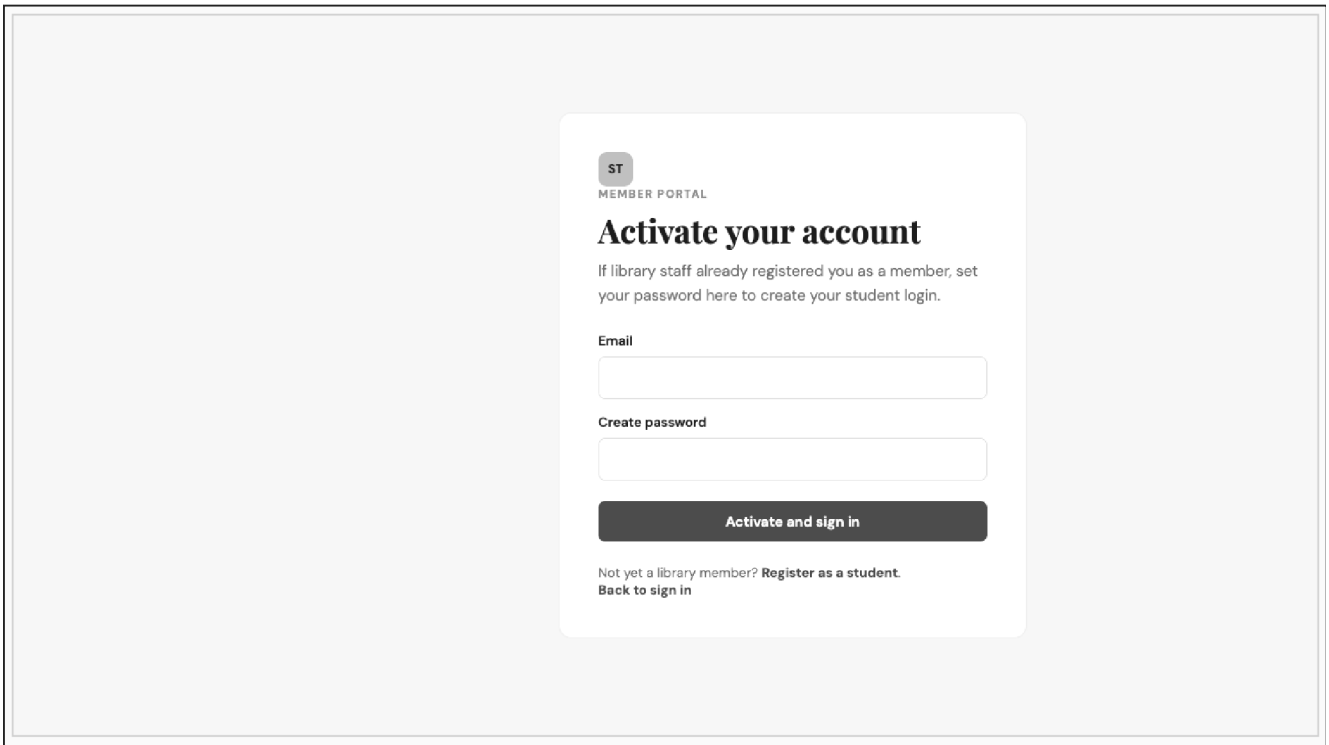


Figure 3.3: Account Activation for Desk-Registered Members

Allows members registered in person by library staff to activate their student online account and establish their personal password.

3.4 Staff Overview & KPI Dashboard (`#dashboard`)

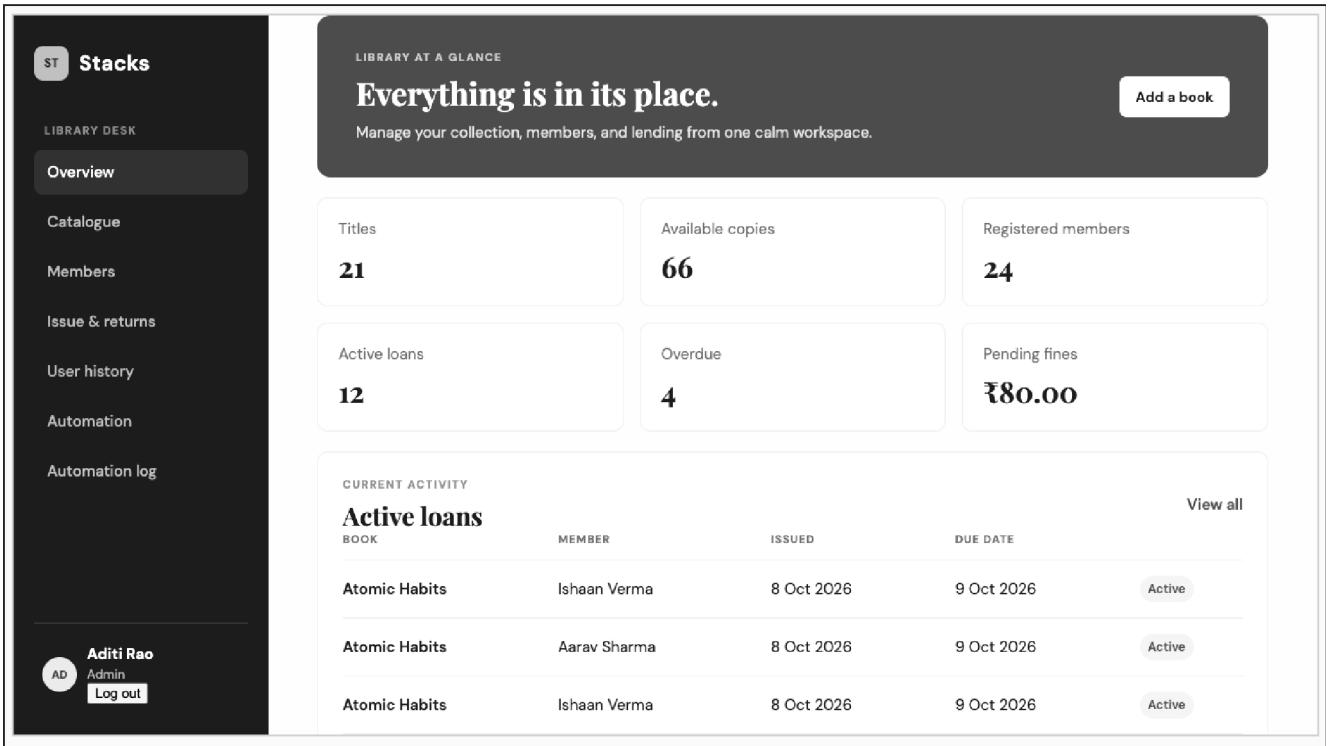


Figure 3.4: Main Administrative Operations Dashboard

Displays high-level KPI cards (titles, available copies, active loans, overdue counts, pending fines) and recent active lending records.

3.5 Book Catalogue (#books)

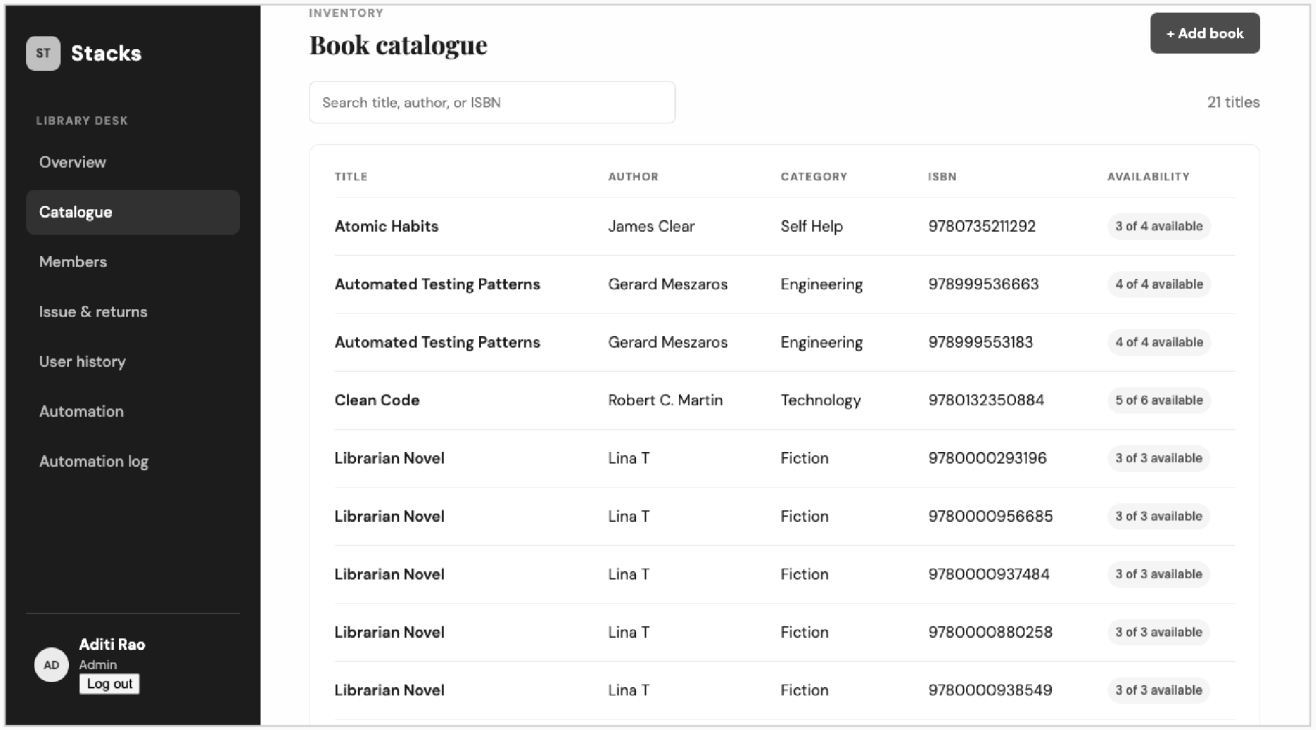


Figure 3.5: Searchable Book Inventory Catalogue

Lists all library titles with real-time text search filtering across title, author, category, ISBN, and stock availability badges.

3.6 Add Book Modal Dialog

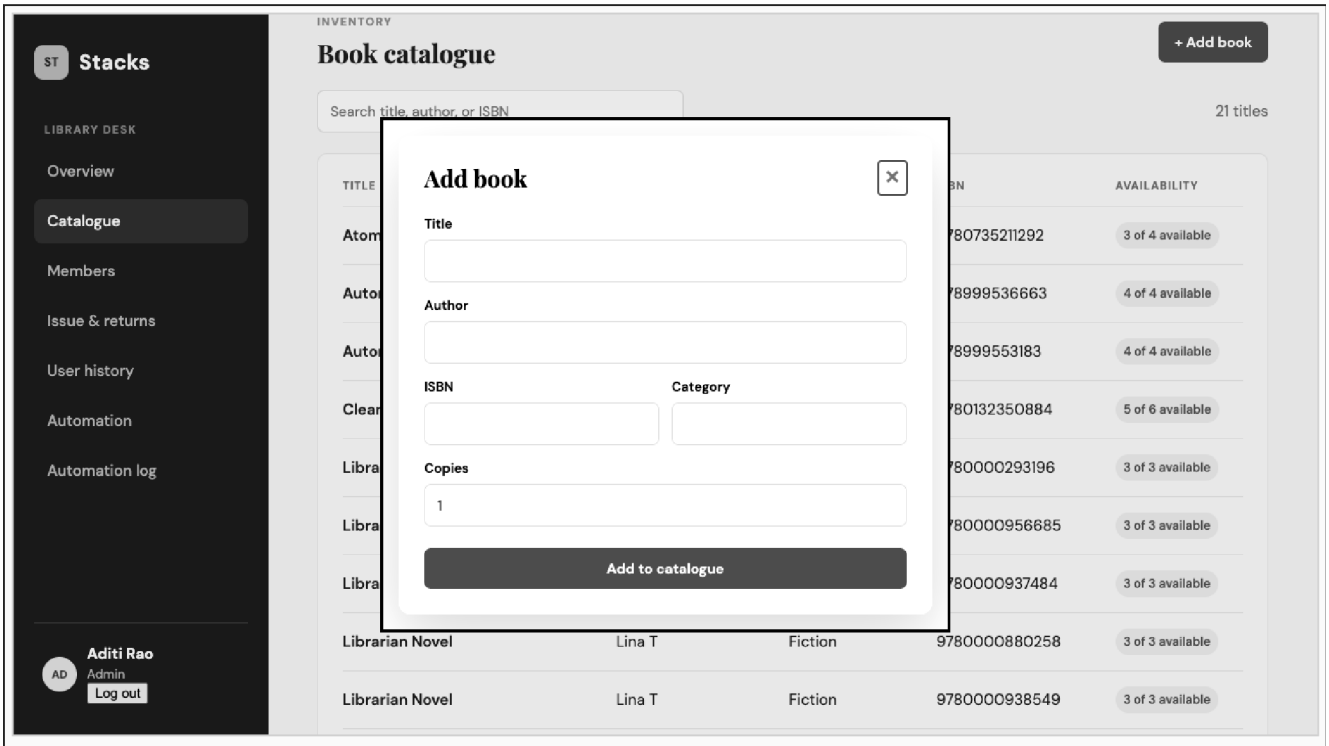


Figure 3.6: Catalogue Book Acquisition Form

Staff input modal ensuring complete book data validation and unique ISBN verification before writing to the database.

3.7 Member Directory (`#members`)

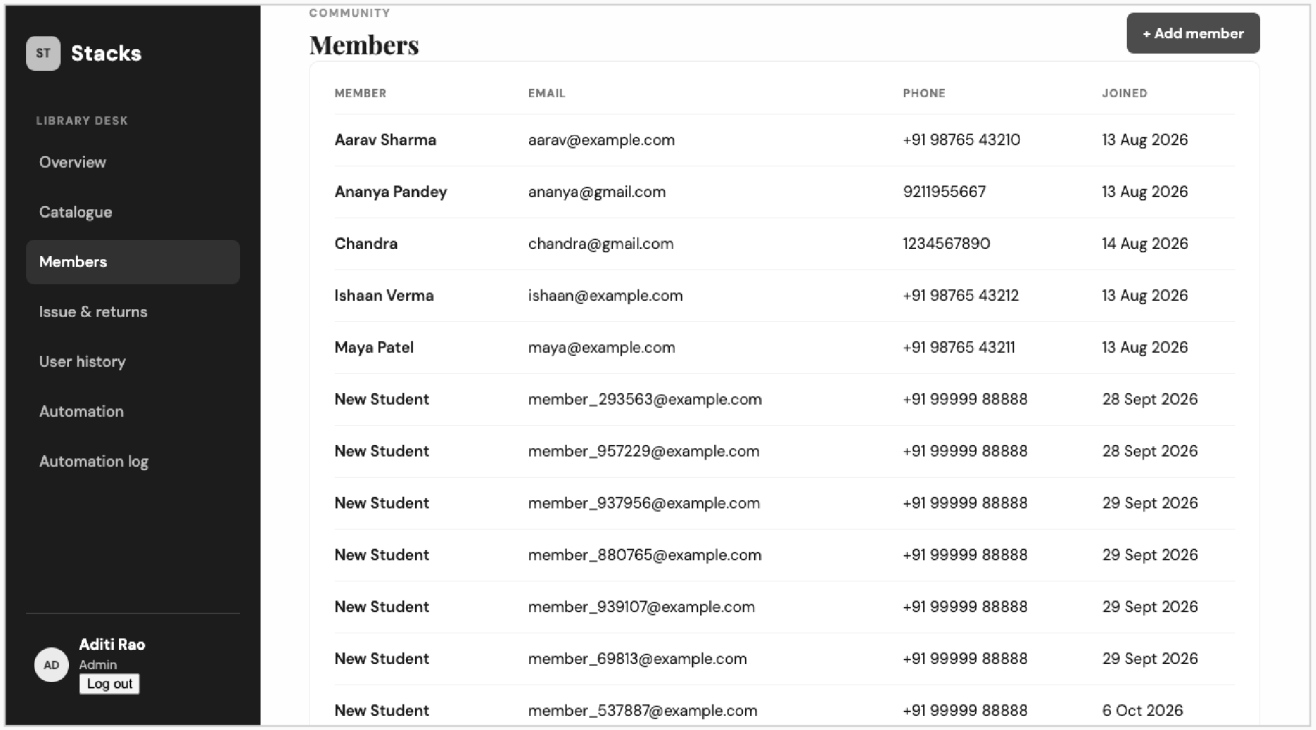


Figure 3.7: Registered Library Member Directory

Staff interface displaying all registered patrons, contact details, join dates, and portal account activation statuses.

3.8 Add Member Modal Dialog

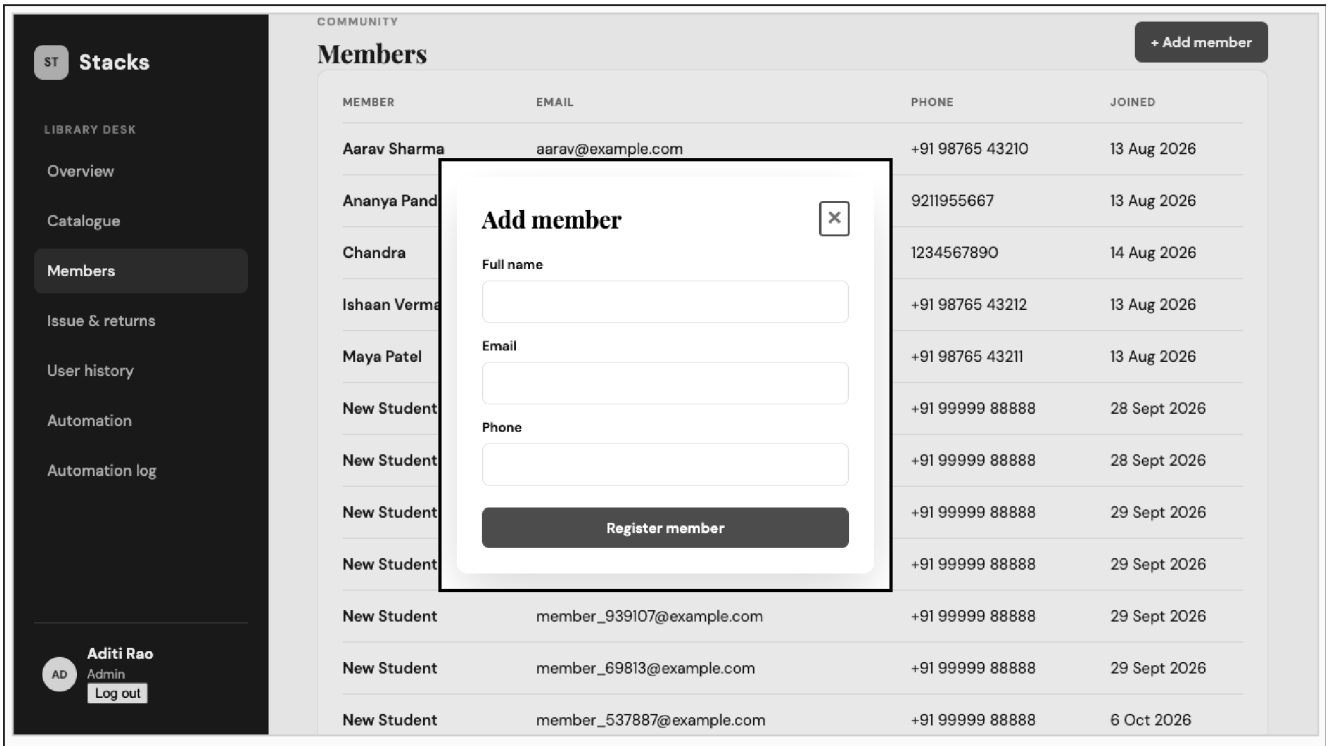


Figure 3.8: In-Person Member Registration Form

Desk modal enabling staff to register patrons directly with mandatory name and email fields.

3.9 Lending Desk: Issues & Returns (`#loans`)

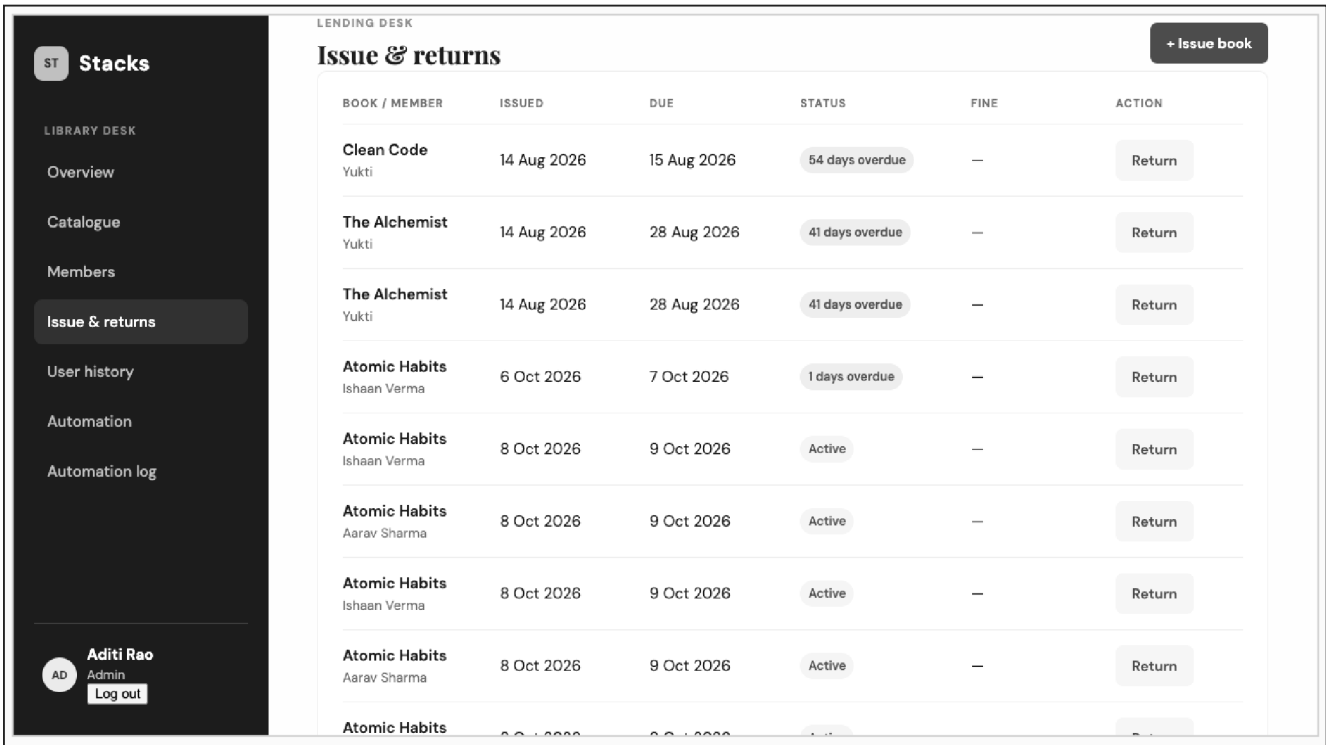


Figure 3.9: Active & Historical Circulation Desk

Main operational desk for managing book borrowings, showing active loans, overdue status tags, return buttons, and fine payment triggers.

3.10 Issue Book Modal Dialog

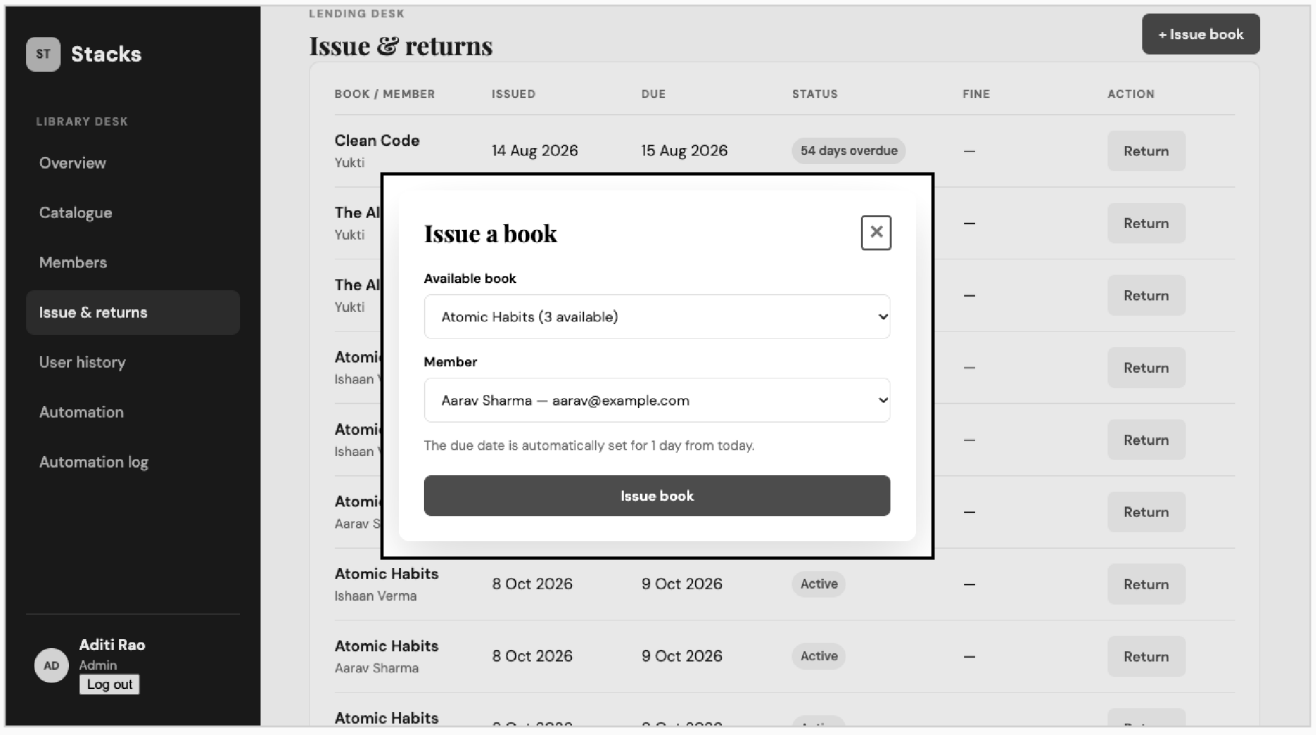


Figure 3.10: Book Loan Checkout Dialog

Dropdown selection dialog matching available books with registered members to create instant loan transactions.

3.11 User Activity History / Staff Audit Trail (`#activity-log`)

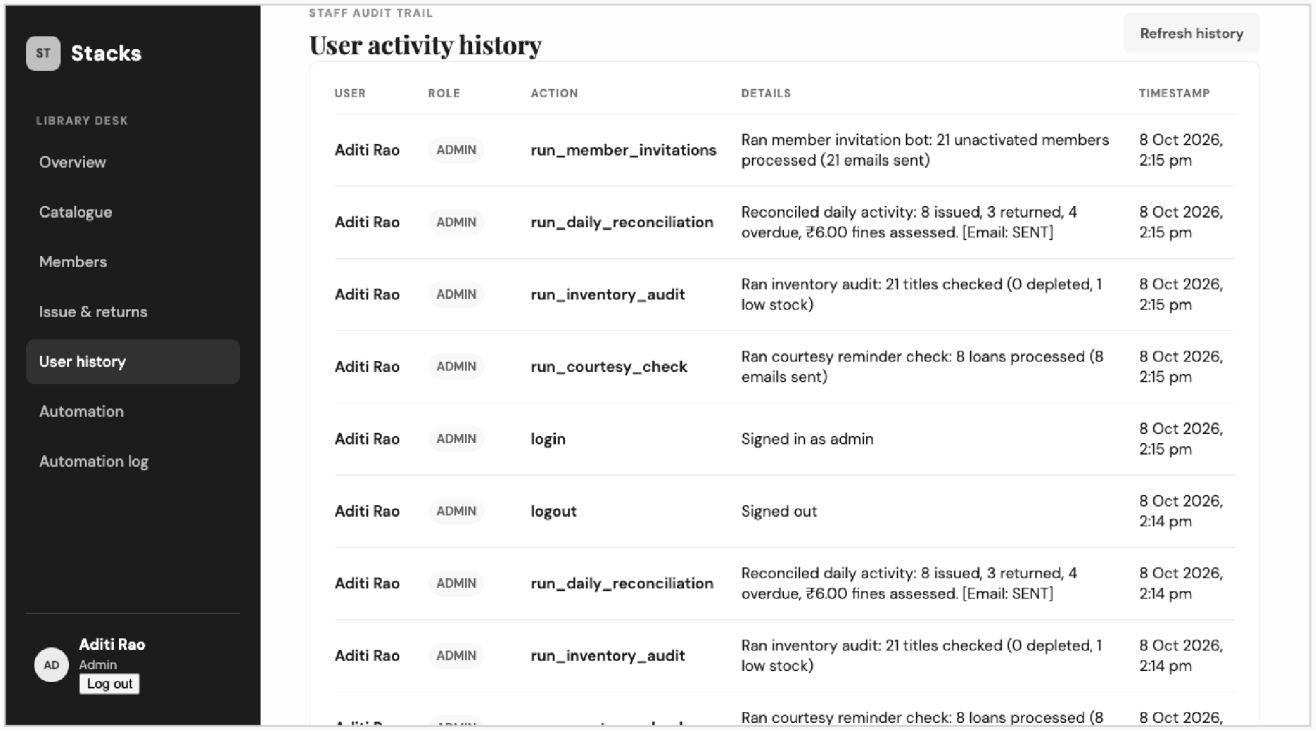


Figure 3.11: Comprehensive Staff Audit Trail

Immutable record tracking all administrative and librarian actions (logins, logouts, book additions, member creations, loan checkouts, and bot triggers).

3.12 Student Portal View

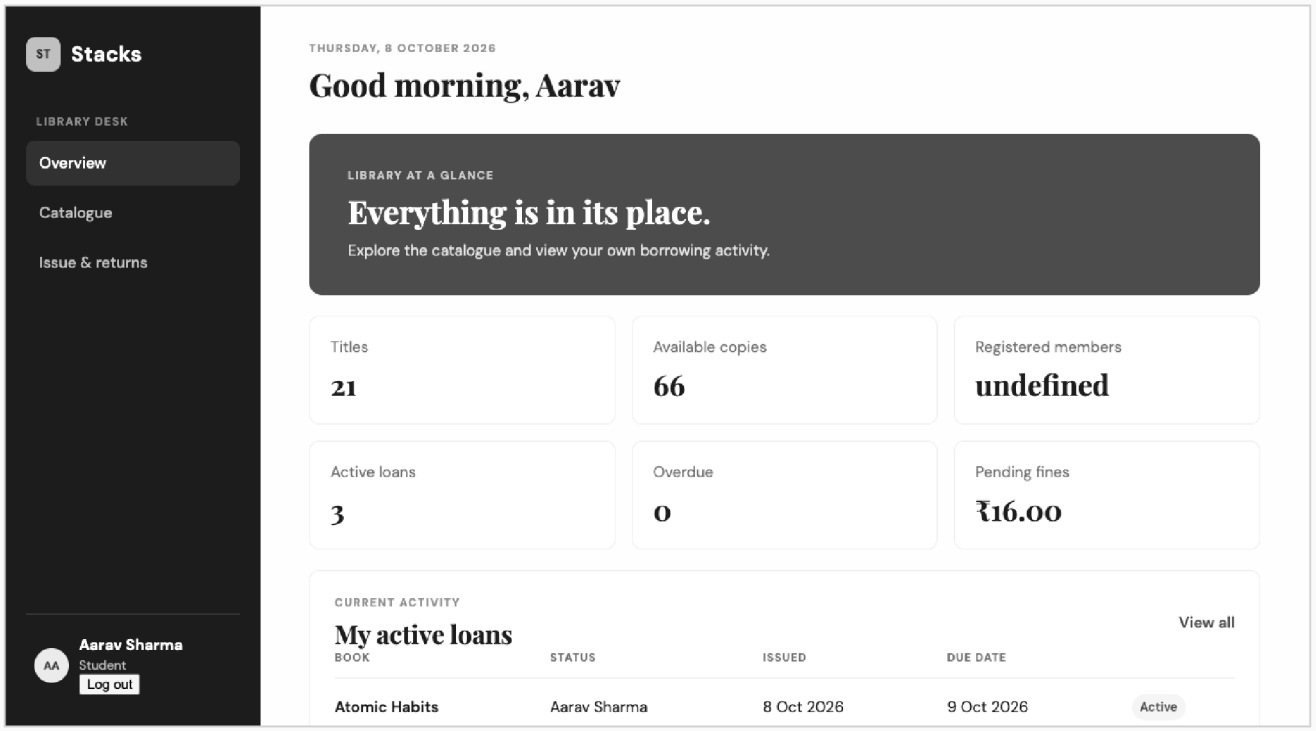


Figure 3.12: Student Self-Service Dashboard

Restricted view for students showing only their active borrowings, return deadlines, and personal fine obligations with administrative controls hidden.

4. RPA Automation Features & Technical Deep Dive

This project incorporates dedicated Robotic Process Automation (RPA) workflows designed to execute both on-demand from the UI and autonomously in scheduled background routines via **Robot Framework**.

4.1 The Automation Centre (`#automation`)

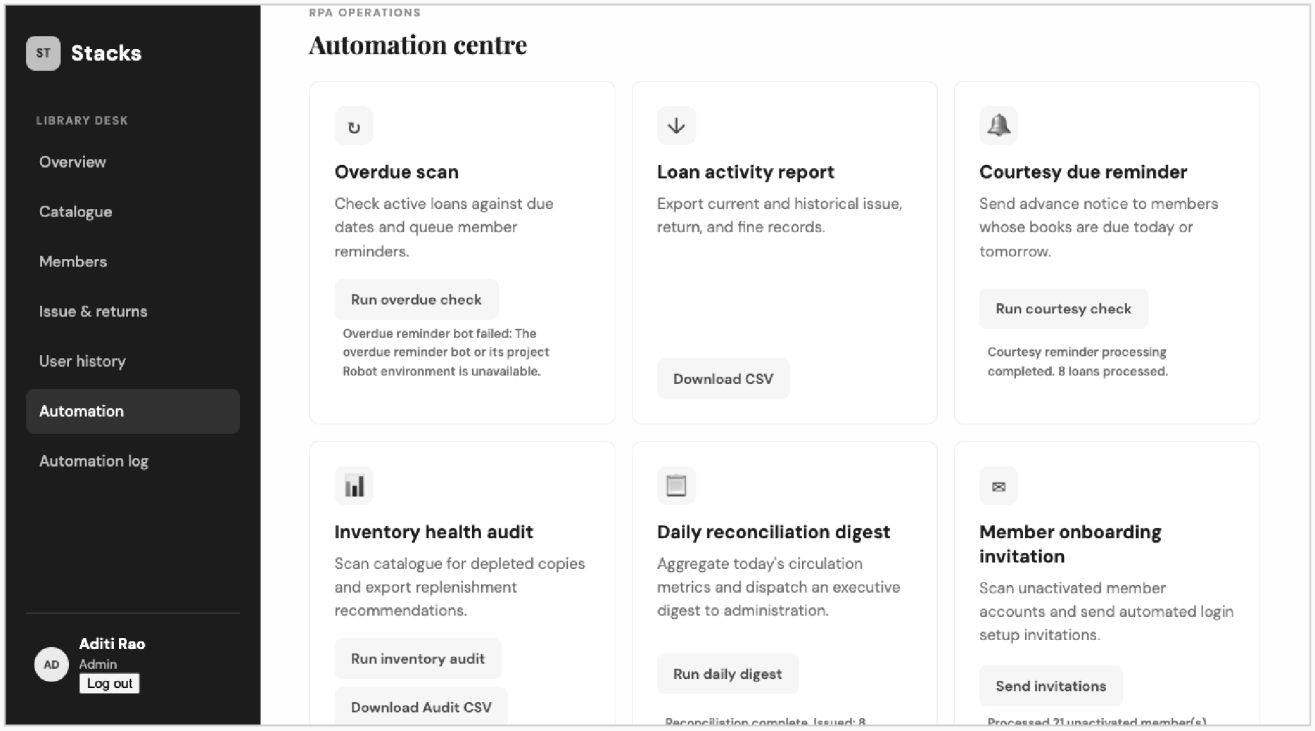


Figure 4.1: Administrative Automation Centre with Bot Triggers & Status Feedback

Centralized hub where administrators can execute automated scans, observe live progress feedback, and trigger export routines.

Automation Feature 1: Daily Overdue Loan Detection & Fine Calculation Bot

Primary Function: Scans active borrowings where `due_on < today`, computes accrued overdue days, calculates penalties at ₹2.00/day, dispatches email reminders, and logs execution records.

Underlying Mechanism: Endpoint `POST /api/automation/overdue` and Robot Framework suite `rpa/bots/overdue_reminder_bot.robot`.

Fault Resilience: SMTP network failures are captured as `[Email: FAILED]` without crashing fine calculations or database logging.

Automation Feature 2: Advance Courtesy Due-Date Reminder Bot

Primary Function: Proactively scans loans due within 24 to 48 hours (`due_on = today + 1 day`) and dispatches courtesy advance notice emails before any penalty is incurred.

Underlying Mechanism: Endpoint `POST /api/automation/courtesy` and Robot suite `rpa/bots/courtesy_reminder_bot.robot`.

Database Logging: Appends directly to `automation_logs` with `overdue_days = 0`, `fine = 0.0`, and message tag `[Email: SENT]`.

4.2 The Automation Log Interface (`#automation-logs`)

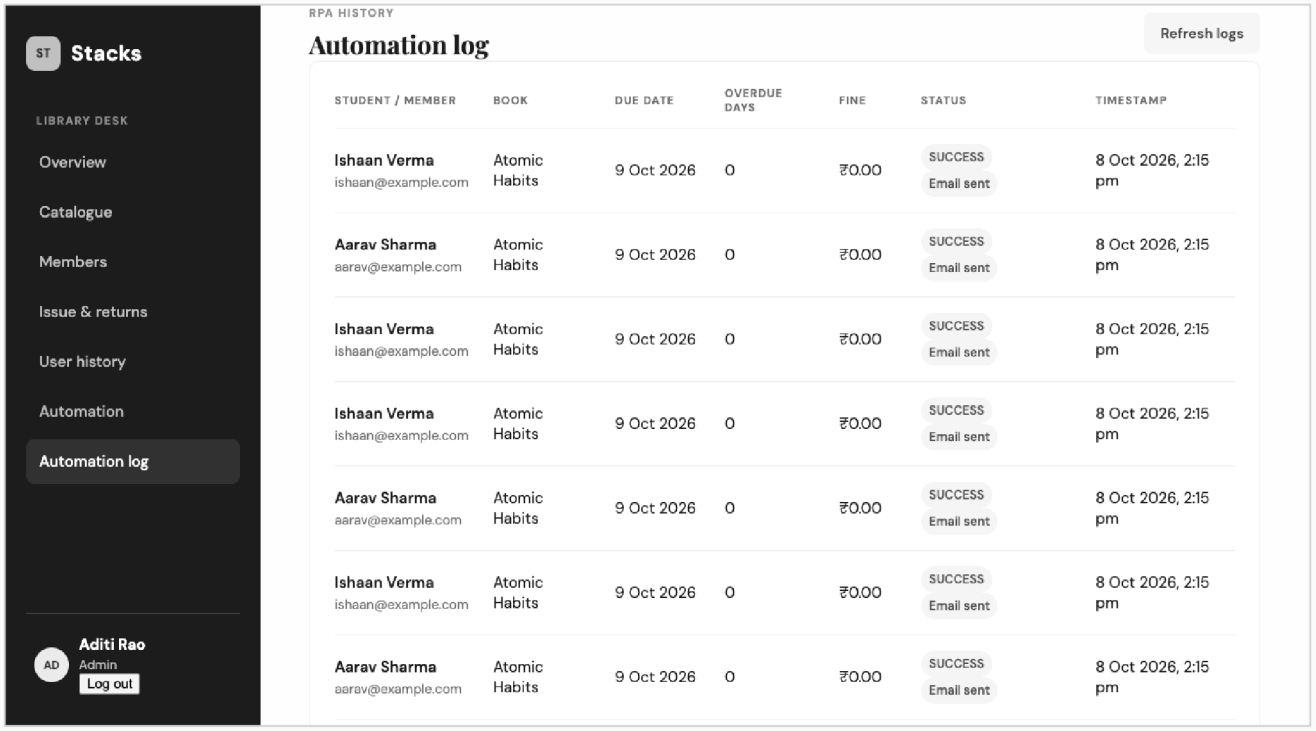


Figure 4.2: Granular Bot Execution History with Status Badges

Displays individual loan actions processed by the software bots, including member details, calculated fines, status tags, and email transmission indicators.

Automation Feature 3: Catalogue Inventory Health & Restock Advisory Bot

Primary Function: Performs an automated audit of the entire book collection, evaluating real-time inventory against borrowing velocity.

Key Metrics Computed:

- `available_copies` vs `total_copies`.
- `utilization_pct`: Percentage of copies currently in circulation.
- `Stock Status`: Classified into `CRITICAL_DEPLETED` (0 available), `LOW_STOCK` (1 available), or `HEALTHY`.
- `Recommended Reorder`: Suggested quantity of copies to purchase based on active borrow demand.

Deliverable: Generates an exportable replenishment advisory report at `GET /api/reports/inventory-audit.csv`.

Automation Feature 4: Daily Managerial Digest & Financial Reconciliation Bot

Primary Function: At the close of each operational day, aggregates total books issued, returns processed, fines assessed, outstanding balances, and active borrowings. Automatically compiles an executive summary digest and emails it to the library administration (`LIBRARY_ADMIN_EMAIL`).

Underlying Mechanism: Endpoint `POST /api/automation/daily-reconciliation` and Python `email_service.send_reconciliation_digest_email`.

Business Impact: Replaces manual spreadsheet end-of-day reconciliation with an automated, auditable financial digest.

Automation Feature 5: Unactivated Member Account Invitation Bot

Primary Function: Scans patrons registered at the desk whose online accounts have not yet been activated (`account_active == 0`). Automatically dispatches personalized onboarding invitation emails directing members to `/activate`.

Underlying Mechanism: Endpoint `POST /api/automation/member-invitations` and Python `email_service.send_activation_invitation_email`.

Business Impact: Streamlines student digital onboarding without requiring manual email correspondence by desk staff.

Automation Feature 6: Bulk Catalogue CSV Ingestion Bot

Primary Function: Replaces manual one-by-one book entry with batch data-entry automation. Parses uploaded CSV files, validates required attributes (Title, Author, ISBN, Category, Copies), gracefully skips malformed rows with diagnostic reasons, deduplicates by ISBN (automatically incrementing existing stock copies), and adds new acquisitions directly to the catalogue. Provides a downloadable standard CSV template for staff convenience.

Underlying Mechanism: Endpoints `POST /api/automation/books/import-csv` and `GET /api/templates/books-template.csv`.

Business Impact: Eliminates hundreds of keystrokes during new semester acquisitions and prevents duplicate ISBN conflicts automatically.

Automation Feature 7: Automated Circulation & Lending History CSV Export

Primary Function: Extracts all transaction history (book titles, members, issue dates, due dates, return dates, assessed fines, and payment records) directly into structured CSV (`GET /api/reports/loans.csv`) for audit archiving.

Automation Feature 8: Dual-Tier Separate Audit Architecture

Primary Function: Maintains strict separation between bot execution telemetry (`automation_logs`) and human staff audit tracking (`user_activity_log`), preventing log pollution and maintaining strict compliance auditing.

5. Role-Based Access Control (RBAC) Matrix

The system enforces strict multi-role permission gates via the `@login_required` decorator across all application views and API endpoints:

Feature / Endpoint	Admin Role	Librarian Role	Student Role	Unauthenticated
View Catalogue & Personal Loans	Allowed	Allowed	Allowed	Redirect to Login (401)
Add Books & Edit Catalogue	Allowed	Allowed	Forbidden (403)	Denied (401)
View Member Directory	Allowed	Allowed	Forbidden (403)	Denied (401)
Issue Books & Process Returns	Allowed	Allowed	Forbidden (403)	Denied (401)
Mark Fines as Paid	Allowed	Allowed	Forbidden (403)	Denied (401)
Run Inventory Audit & Download CSV	Allowed	Allowed	Forbidden (403)	Denied (401)
Bulk Catalogue CSV Ingestion & Template	Allowed	Allowed	Forbidden (403)	Denied (401)
Trigger Member Invitation Bot	Allowed	Allowed	Forbidden (403)	Denied (401)
Trigger Daily Reconciliation Digest Bot	Allowed	Forbidden (403)	Forbidden (403)	Denied (401)
Trigger Overdue & Courtesy RPA Bots	Allowed	Forbidden (403)	Forbidden (403)	Denied (401)
View Staff Audit Trail (User History)	Allowed	Allowed	Forbidden (403)	Denied (401)
View Automation History Logs	Allowed	Forbidden (403)	Forbidden (403)	Denied (401)

6. Quality Assurance & Test Verification Summary

All system capabilities and automation bots are backed by automated unit tests and Robot Framework verification suites:

Test Suite	Coverage Area	Test Count	Status
`test_milestone2.py`	Staff Activity Logging & Role Enforcement	10 Tests	**10 / 10 PASS**
`test_milestone3_email.py`	Email Subsystem, SMTP Resilience & Overdue Integration	9 Tests	**9 / 9 PASS**

Test Suite	Coverage Area	Test Count	Status
`test_milestone4.py`	Courtesy, Inventory, Reconciliation, Onboarding & Bulk CSV	15 Tests	**15 / 15 PASS**
`test_milestone3.py` (Legacy)	Live Server Milestone Verifier & Schema Validation	50 Checks	**50 / 50 PASS**
`courtesy_reminder_bot.robot`	Robot Framework Courtesy Bot API Automation	1 Scenario	**1 / 1 PASS**
`overdue_reminder_bot.robot`	Robot Framework Overdue Bot API Automation	1 Scenario	**1 / 1 PASS**

Overall Test Result: 100% Pass Rate across all 34 unit tests and 50 live verification assertions with zero regressions.